Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Bu ders tümüyle öğrenci sunumlarından oluşur
  - Sunumlar mikroservis mimarisinde test (birim, entegrasyon, bileşen, uçtan uca), üretime hazır servisler (güvenlik, yapılandırılabilirlik, gözlemlenebilirlik, mikroservis şasisi) ve AWS Lambda ile servis dağıtımı konularını işler.
  - Birkaç sunum İngilizcedir ve altyazının bu kısımları büyük ölçüde bozuktur. Bu kısımlar ekrandaki slaytlardan tamamlanmıştır.
- Mikroservislerde test stratejileri
  - Test, test edilen sistemin (SUT, System Under Test) davranışını doğrulamak içindir. SUT tek bir sınıftan bütün uygulamaya kadar değişebilir.
  - Otomatik bir test dört aşamadan oluşur: Setup, Exercise, Verify, Teardown (ekrandan, 00:03:40).
  - Dört test türü: birim (unit), entegrasyon, bileşen (component) ve uçtan uca (end-to-end) testler (ekrandan, 00:06:00).
  - Test piramidi: alttan üste Unit, Integration, Component, End-to-end. Yukarı çıkıldıkça testler yavaş, kırılgan ve pahalı olur; bu yüzden çok sayıda birim testi, az sayıda uçtan uca test yazılır (ekrandan, 00:08:00).
  - Tüketici güdümlü sözleşme testi (consumer-driven contract testing): sağlayıcının API'sinin tüketicinin beklentilerini karşıladığını sözleşme denen örneklerle doğrulayan bir entegrasyon testidir (ekrandan, 00:10:00).
- Servisler için birim testleri
  - "Birim" tanıma göre değişir: nesneye yönelik tasarımda genelde bir sınıf, fonksiyonel programlamada bir fonksiyon ya da metottur.
  - Birim testleri çok hızlı (milisaniyeler) ve izole çalışmalıdır. Jest, JUnit, pytest, unittest gibi çerçevelerle yazılır.
  - Solitary (yalnız) birim testi bağımlılıkların yerine test double (mock, stub) kullanır; sociable (sosyal) birim testi bağımlılıkları gerçek haliyle çalıştırır.
  - Örnek: `MyUnit.concatenate("one", "two")` sonucunun `"onetwo"` olduğu `assertEquals` ile doğrulanır (ekrandan, 00:20:00).
  - Entity (`Order.getOrderTotal`) için sociable, controller (`OrderController`) için solitary test örnekleri verildi.
- Servisler için entegrasyon testleri
  - Bir servisin veritabanı, mesajlaşma ve diğer servislerle doğru etkileşime girdiğini doğrular; birim testlerinin bir üst katmanıdır.
  - İki strateji: her adaptörü destekleyici sınıflarıyla birlikte test etmek ve sözleşmeler (contract) kullanmak.
  - Üç etkileşim türü ve sözleşmeleri: REST tabanlı istek/yanıt (HTTP istek ve yanıtı), publish/subscribe (domain event), asenkron istek/yanıt (komut ve yanıt mesajı).
  - Kalıcılık (persistence) entegrasyon testi: kurulum, yürütme, doğrulama ve isteğe bağlı teardown.
- Bileşen testleri
  - Bir servisin kabul testlerini, servisin bağımlılıklarını stub ile değiştirerek servisi yalıtılmış biçimde test eden testlerdir.
  - Kabul testleri kullanıcı hikâyelerinden üretilir; Gherkin ile Given/When/Then senaryoları yazılır, Cucumber ile çalıştırılır.
  - Süreç içi (in-process) ve süreç dışı (out-of-process) bileşen testleri karşılaştırıldı; Doc-

ker Compose ve Gradle ile çalıştırma anlatıldı.
- Uçtan uca testler
  - Bütün uygulamanın akışını baştan sona test eder; test piramidinin tepesindedir.
  - Kullanıcı yolculuğu (user journey) testleri az sayıda ama kapsayıcı yazılmalıdır.
- Güvenli servis geliştirme
  - Monolitin bellek içi güvenlik bağlamı ve merkezi oturum yaklaşımı mikroservislere uymaz.
  - Kimlik doğrulama ve yetkilendirme API Gateway'de merkezileştirilir; kullanıcı kimliği servisler arasında JWT ile taşınır; OAuth 2.0 kullanılır.
- Yapılandırılabilir servis tasarımı
  - Dışsallaştırılmış yapılandırma (externalized configuration): push tabanlı ve pull tabanlı modeller.
  - Spring Cloud Config: yapılandırma sunucusu, yapılandırma deposu (veritabanı, dosya sistemi ya da Git) ve yapılandırma istemcisi.
- Gözlemlenebilir servis tasarımı
  - Health check API, log aggregation, distributed tracing, exception tracking, application metrics ve audit logging desenleri.
- Mikroservis şasisi (microservice chassis) ve service mesh
  - Ortak kesişen kaygıları (cross-cutting concerns) her serviste tekrar yazmamak için ortak bir çerçeve kullanılır.
- AWS Lambda ve API Gateway ile RESTful servis dağıtımı
  - Lambda fonksiyonu oluşturma, API Gateway'de REST API, kaynak (resource) ve metot tanımlama, deploy ve Invoke URL ile çağırma adım adım gösterildi.

Hocanın Özellikle Vurguladığı Kısımlar

- Konular birbirine bağlıdır
  - Hoca sunuculardan, önceki sunumda anlatılan kısımlar üzerinde tekrar durmamalarını istedi. Bu yüzden test sunumları birbirinin devamı gibi ilerler.
- Uçtan uca test konusunun kısalığı
  - Hoca uçtan uca test sunumunun kısa olduğunu, çünkü Gradle ve Docker Compose ile çalıştırma kısmının önceki bileşen testi sunumunda zaten anlatıldığını belirtti.
- AWS Lambda sunumunun yeri
  - Hoca bu sunumun aslında dağıtım (deployment) haftasının konusu olduğunu, uygulamaya dönük olduğu için bu haftaya alındığını söyledi.
- Bu derste hocanın ayrıca anlattığı akademik bir konu yoktur; hoca sunumları yönetti.

Kısa Tekrar Notları

- Otomatik test aşamaları: Setup → Exercise → Verify → Teardown (teardown isteğe bağlıdır).
- Test piramidi (alttan üste): Unit → Integration → Component → End-to-end. Alt: hızlı, güvenilir, ucuz. Üst: yavaş, kırılgan, pahalı.
- Unit test iş mantığını, integration test servisin bağımlılıklarıyla iletişimini, component test bir servisin kabul testlerini, end-to-end test bütün uygulamanın kabul testlerini kapsar.
- Solitary birim testi: bağımlılıklar mock/stub. Sociable birim testi: bağımlılıklar gerçek.
- Entity sınıfları için sociable, dış çağrı yapan controller ve servisler için solitary test tercih edilir.
- Sözleşme (contract): bir servis çiftinin etkileşimine ait somut örnek. Tüketici tarafı testi sözleşmeyle sağlayıcıyı simüle eder; sağlayıcı tarafı testi sözleşmeyle sağlayıcının adaptörünü test eder.
- Spring Cloud Contract ile REST sözleşmesi: `GET /orders/1223232` → `status 200`, JSON gövde.
- Gherkin: Given / When / Then. Cucumber: Gherkin senaryolarını çalıştıran test çerçevesi; adım tanımları `@Given`, `@When`, `@Then` notasyonlu Java metotlarıdır.
- Süreç içi bileşen testi: bellek içi veritabanı ve stub; basit ve hızlı ama dağıtılabilir servisi test etmez. Süreç dışı bileşen testi: gerçek veritabanı ve mesaj aracısı; kapsamı daha iyi ama karmaşık, yavaş, kırılgan.

- Uçtan uca testler az sayıda yazılmalıdır.
- Mikroservis güvenliğinde kimlik doğrulama ve yetkilendirme API Gateway'de merkezileştirilir. Opaque token performansı ve erişilebilirliği düşürür; JWT ele geçirilirse geri alınamaz. OAuth 2.0 bu sorunu çözen standarttır.
- Push modeli: dağıtım altyapısı yapılandırmayı ortam değişkenleriyle verir. Pull modeli: servis yapılandırmasını başlangıçta yapılandırma sunucusundan okur.
- Metrik örneği üç özellik taşır: ad, değer ve zaman damgası (ekrandaki örnek: `cpu_percent`, `68`).
- Şasi deseni avantajları: iş mantığını kesişen kaygılardan ayırma, yapılandırılabilir temel çerçeve, kararlı kod tabanı, hızlı uygulama geliştirme desteği, özelliği yalnız ihtiyaç duyan servise bağımsız ekleme.
- Service mesh: devre kesici, servis keşfi, güvenli iletişim, akıllı trafik yönlendirme, dağıtık izleme ve yük dengeleme işlevlerini şasiden altyapıya taşır.
- AWS mimari örneği: statik içerik S3'te, iş mantığı Lambda'da, dışa açma API Gateway ile, veritabanı DynamoDB.

Detaylı Açıklamalar

- Test stratejisinin genel çerçevesi
  - İlk sunum, testi yazılım mimarisinin önemli bir parçası olarak sunar: yetersiz test edilmiş bir bina gibi, ihmal edilen bir test aşaması da sonradan önlenemeyen kayıplara yol açabilir. Bir testin amacı SUT'un davranışını doğrulamaktır.
  - Otomatik test, test çalıştırıcının (test runner) yürüttüğü bir test sınıfındaki test metotlarından oluşur. Her test metodu önce fixture'ı istenen başlangıç durumuna getirir (Setup), SUT'u çağırır (Exercise), çağrının sonucu ve SUT'un durumu hakkında iddialarda bulunur (Verify) ve gerekirse fixture'ı temizler (Teardown) (ekrandan, 00:03:40).
  - Test piramidinin temel fikri, piramitte yukarı çıkıldıkça daha az test yazmaktır. Piramidin yan etiketleri katmanların görevini verir: Unit "iş mantığını test et", Integration "servisin bağımlılıklarıyla iletişimini doğrula", Component "bir servisin kabul testleri", End-to-end "bir uygulamanın kabul testleri" (ekrandan, 00:08:00).
  - Tüketici güdümlü sözleşme testinde her tüketici takımı (API Gateway takımı, Order History Service takımı vb.) Order Service için kendi sözleşme test takımını yazar. Bu takımlar Order Service'in dağıtım hattında (deployment pipeline) çalıştırılır. Böylece sağlayıcı, API'sini değiştirdiğinde hangi tüketiciyi bozduğunu dağıtımdan önce görür (ekrandan, 00:10:00).
- Birim testi örnekleri
  - En basit örnekte `MyUnit` sınıfının `concatenate(String one, String two)` metodu `one + two` döndürür. `MyUnitTest` sınıfındaki `@Test` notasyonlu `testConcatenate()` metodu bir `MyUnit` nesnesi oluşturur, `concatenate("one", "two")` çağırır ve `assertEquals("onetwo", result)` ile sonucu doğrular (ekrandan, 00:20:00). Test adının test edilen metoda göre (`testConcatenate`) verilmesi testi kendi kendini belgeleyen bir hale getirir.
  - Metot `two`'yu eklemeyi unutursa test başarısız olur ve çerçeve beklenen ile gerçek değeri gösterir ("expected: onetwo, actual: one" gibi). Bu, hatanın yerini bulmaya yardım eder.
  - `assertTrue`, `assertFalse`, `assertNotEquals` gibi başka assertion metotları da vardır. Jest'te `expect` ile `toBe`, `toHaveBeenCalled` gibi eşleyiciler ve `expect(...).not` kullanılır.
  - Controller testi (ekrandan, 00:25:15): `@Before` notasyonlu `setUp()` içinde `OrderService` ve `OrderRepository` `mock(...)` ile oluşturulur, asıl test edilen `OrderController` gerçek nesne olarak kurulur. `shouldFindOrder()` testinde `when(orderRepository.findById(1L))` ile mock yapılandırılır; ardından REST Assured ile `get("/orders/1")` isteğinin `statusCode(200)` döndürdüğü ve gövdedeki `orderId` değerinin beklenen değere eşit olduğu doğrulanır.
  - Sunucu birim testlerinin artısını ve eksisini şöyle özetledi: hızlı çalışır, kolay yazılır, hataları erken bulur; ama birimler arasındaki ilişkileri kavramaz, bu ilişkilerdeki bir bozulmayı gözden kaçırabilir. Birim testleri test güdümlü geliştirmede (TDD) koddan önce yazılırsa mimari için bir harita ve belge işlevi de görür.
- Entegrasyon testi ayrıntıları

  - Order Service örneğinde API Gateway (tüketici) Order Service'in `OrderController`'ına REST ile, Order History Service olay kanalı üzerinden `Order aggregate event publisher`'a, `Kitchen Service proxy` ise komut ve yanıt kanallarıyla Kitchen Service'e bağlıdır. Her bağlantıda test edilen sınıf ayrı bir testle sınanır; `OrderRepository` de veritabanıyla birlikte test edilir (ekrandan, 00:30:00).
  - Kalıcılık entegrasyon testi şu aşamalardan oluşur: veritabanı şemasını oluşturup bilinen bir duruma getirme ve bir veritabanı işlemi (transaction) başlatma; işlemi yürütme; veritabanının durumu ve veritabanından okunan nesneler hakkında iddialarla doğrulama; kurulumda başlatılan işlemi geri alarak değişiklikleri temizleyen isteğe bağlı teardown.
  - REST sözleşmesi örneği (ekrandan, 00:35:15):

```
org.springframework.cloud.contract.spec.Contract.make {
    request {
        method 'GET'
        url '/orders/1223232'
    }
    response {
        status 200
        headers {
            header('Content-Type': 'application/json;charset=UTF-8')
        }
        body('''{"orderId" : "1223232", "state" : "APPROVAL_PENDING"}''')
    }
}
```

- Publish/subscribe etkileşiminde sözleşme bir domain event'tir; tüketici olayın yapısı ve kanalı üzerinde sağlayıcıyla anlaşır. Asenkron istek/yanıtta ise taraflar komut kanalının adı ile komut ve yanıt mesajlarının yapısı üzerinde anlaşır; testler için komut ve yanıt kanalları tanımlanır.
- Entegrasyon testleri bütün sistemi çalıştırmaz, yalnız servisler arası etkileşimleri sınar; bu yüzden uçtan uca testlerden hızlıdır.
- Bileşen testi ayrıntıları
  - Kabul senaryosu örneği: geçerli bir tüketici, geçerli bir kredi kartıyla, siparişleri kabul eden bir restorandan (Ajanta) Chicken Vindaloo sipariş ettiğinde sipariş onaylanmalı ve bir `OrderAuthorized` olayı yayınlanmalıdır. Süresi dolmuş kartla verilen siparişte ise sipariş reddedilmeli ve `OrderRejected` olayı yayınlanmalıdır.
  - Senaryo için iki çağrı yapılır: `POST /orders` ile sipariş oluşturulur, `GET /orders/{orderId}` ile siparişin durumu doğrulanır.
  - Cucumber adım tanımları (ekrandan, 00:43:25) `@Given("A valid consumer")`, `@Given("using a(.?) (.*) credit card")`, `@When("I place an order for Chicken Vindaloo at Ajanta")`, `@Then("the order should be (.*)")` ve `@And("an (.*) event should be published")` notasyonlu metotlardır. Notasyondaki düzenli ifade grupları metot parametrelerine aktarılır.
  - `useCreditCard` adımı (ekrandan, 00:46:15) kart `"valid"` ise `sagaParticipantStubManager.forChannel("accountingService").when(AuthorizeCommand.class).replyWithSuccess()`, `"invalid"` ise `replyWithFailure()` ile accounting servisinin stub'ını yapılandırır; başka bir değer gelirse `fail("Don't know what to do with this credit card")` ile testi düşürür.
  - Süreç içi bileşen testi H2, HSQLDB ya da Derby gibi bellek içi veritabanları ve bellek içi stub'larla çalışır. Süreç dışı bileşen testi gerçek altyapıyı (veritabanı, mesaj aracısı) kullanır, uygulama servisleri için stub kullanır.
  - Testler Docker ile çalıştırılır. Gradle Docker Compose eklentisi bileşen testlerinden önce servisi ve altyapı servislerini başlatır; testlerden önce Docker imajının ihtiyaç duyduğu JAR dosyası oluşturulur. Tek bir komut servisi derler, servisi ve altyapıyı başlatır, testleri çalıştırır ve servisleri durdurur.
- Uçtan uca testler
  - Uçtan uca testlerde çok sayıda hareketli parça vardır; çok sayıda servis ve altyapının dağıtılması gerekir. Bu yüzden yavaştırlar ve bir servisin başlamaması testi güvenilmez kılabilir (ekrandan, 00:50:00).
  - Kullanıcı yolculuğu örneği: geçerli müşteri sipariş verir (sipariş onaylanır, fiyat be-

lirlenir), siparişi iki ürün daha ekleyerek revize eder (fiyat güncellenir) ve sonunda iptal eder (iptal onayı görülür). Bu senaryo da Gherkin ile yazılır.
  - Sunucuya göre örnek sipariş uygulamasının uçtan uca testleri yaklaşık 4-5 dakika sürmüştür; büyük uygulamalarda bu süre çok daha uzundur.
- Güvenlik
  - Mikroservisler bileşenleri değiştirmeyi ve birden çok geliştiricinin birbirini etkilemeden çalışmasını kolaylaştırır, ama kendine özgü güvenlik zorlukları getirir (ekrandan, 00:55:30).
  - Monolitin iki güvenlik öğesi mikroservislerle uyumsuzdur: bellek içi güvenlik bağlamı (servisler belleği paylaşmaz, kimliği servisten servise taşımak için başka bir mekanizma gerekir) ve merkezi oturum (servislerin ortak bir oturum veritabanına erişmesi gevşek bağlılık ilkesini çiğner).
  - API Gateway istekleri servislere iletmeden önce doğrular. Böylece güvenlik açığı olasılığı azalır ve farklı kimlik doğrulama mekanizmalarıyla yalnız gateway uğraşır (ekrandan, 00:58:30). Yetkilendirmede kullanıcı bir servise erişemiyorsa gateway isteği servise iletmeden reddedebilir (ekrandan, 01:00:00).
  - İki token türü vardır: opaque token (performansı ve erişilebilirliği düşürür, gecikmeyi artırır) ve JWT (kullanıcı kimliği ve rolleri gibi iddiaları iki taraf arasında güvenle taşır; ama kötü niyetli birinin eline geçerse geri alınamaz). Bu sorun OAuth 2.0 standardıyla çözülür (ekrandan, 01:02:00).
- Yapılandırma
  - Bir servis, çalıştığı ortama göre değeri değişen yapılandırma özelliklerine (mesaj aracısı, veritabanı bağlantıları vb.) ihtiyaç duyar. Bu özellikler çalışma anında dışarıdan verilmelidir.
  - Push modelinde dağıtım altyapısı servis örneğini oluştururken ortam değişkenlerini (`BOOTSTRAP_SERVERS=kafka1:9092`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`) ayarlar; servis bunları başlangıçta okur (ekrandan, 01:08:30).
  - Pull modelinde servis başlangıçta yapılandırma sunucusuna `getConfiguration("orderHistoryService")` gibi bir sorgu gönderir. Sunucunun adresi (`CONFIG_SERVER_URL`) ise push tabanlı bir mekanizmayla, örneğin ortam değişkeniyle verilir (ekrandan, 01:10:00).
  - Yapılandırma sunucusunun yararları: bütün özellikler tek yerde tutulur; global varsayılanlar tanımlanıp servis bazında ezilebilir; hassas veriler şifrelenebilir ve bazı sunucular bunları otomatik çözer.
  - Spring Cloud Config, yapılandırma sunucusu, yapılandırma deposu (veritabanı, dosya sistemi ya da Git) ve Spring tabanlı servislerin HTTP ile konuştuğu yapılandırma istemcisinden oluşur. Sunucunun iskeleti Spring Boot Initializr ile alınabilir; kaynak olarak yeni bir Git deposu oluşturulur.
- Gözlemlenebilirlik
  - Amaç uygulamanın ne yaptığını bilmek (saniyedeki istek sayısı, kaynak kullanımı) ve bir sorun olduğunda (çöken servis örneği, dolan disk) uyarı alıp kök nedeni bulabilmektir (ekrandan, 01:17:00).
  - Health check API: servis, isteği karşılayıp karşılayamayacağını altyapıya bildiren bir uç nokta sunar.
  - Log aggregation: her servis örneğinin günlükleri bir log pipeline üzerinden bir günlük sunucusunda toplanır; kullanıcı burada arar ve uyarı alır (ekrandan, 01:20:00).
  - Distributed tracing: her dış isteğe benzersiz bir kimlik atanır ve isteğin servisler arasındaki akışı izlenir. Exception tracking: istisnalar bir istisna izleme servisine raporlanır.
  - Application metrics: servisin içindeki metrik kütüphanesi örnekleri metrik servisine gönderir; metrikler altyapı düzeyinde (CPU, bellek, disk) ya da uygulama düzeyinde (istek gecikmesi, istek sayısı) olabilir (ekrandan, 01:23:00).
  - Audit logging: kullanıcının kimliği, yaptığı eylem ve ilgili iş nesnesi kaydedilir, genelde bir veritabanı tablosunda tutulur. Üç uygulama yolu: iş mantığına eklemek, AOP kullanmak, event sourcing kullanmak.
- Şasi ve service mesh
  - Şasi olmadan her servis şablon derleme mantığını ve şablon kesişen kaygıları tekrar eder (ekrandan, 01:28:30). Şasi bu ortak kısımları bir kez yazılıp paylaşılan bir çerçeveye taşır; servise yalnız kendine özgü mantık kalır. Örnek kesişen kaygılar: günlükleme, dağıtık izleme, istisna işleme, güvenlik.

  - Java için Spring Boot ve Spring Cloud şasi çerçevesi olarak anıldı.
  - Service mesh deseninde devre kesici, servis keşfi, güvenli iletişim, akıllı trafik yönlendirme, dağıtık izleme ve yük dengeleme şasiden altyapı katmanına taşınır; şasiye günlükleme, dışsallaştırılmış yapılandırma, health check ve metrikler gibi daha az işlev kalır (ekrandan, 01:31:30).
- AWS Lambda ile RESTful servis
  - AWS konsolunda Lambda sayfasında “Create function” ile sıfırdan bir fonksiyon oluşturulur; fonksiyon adı, çalışma zamanı (sunumda Node.js, Python da seçilebilir) ve mimari seçilir.
  - Sunucunun örnek fonksiyonu bir kullanıcı listesini 200 durum koduyla döndürür.
  - Fonksiyonu dışarı açmak için API Gateway’de “REST API → Build” seçilir, “New API” ile ad ve açıklama verilir. “Actions → Create Resource” ile `users` kaynağı, “Create Method” ile bu kaynağa bir metot eklenir; entegrasyon türü olarak Lambda Function, bölge ve fonksiyon adı (`GetUsers`) seçilir. “Deploy API” ile yayına alınır ve verilen Invoke URL çağrıldığında Lambda’nın döndürdüğü kullanıcı listesi gelir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.